Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт
З ЛАБОРАТОРНОЇ РОБОТИ № 8
з дисципліни «Професійний Python»
на тему: «Мережеве та асинхронне програмування. REST API»

Виконав:
студент 3 курсу
групи ФеП-33
Франков Дмитро
Олександрович
Перевірив:
професор Свелеба Сергій Андрійович

Львів-2026

1. ТЕМА ТА МЕТА РОБОТИ

Тема роботи: Мережеве та асинхронне програмування. REST API.

Мета роботи: Ознайомитися з основами мережевого програмування у Python, принципами функціонування клієнт-серверної архітектури, протоколу передачі гіпертексту HTTP, архітектурного стилю REST та формату обміну даними JSON. Навчитися проєктувати та створювати високопродуктивні асинхронні REST API за допомогою сучасного веб-фреймворку FastAPI, описувати та суворо валідувати структури запитів і відповідей за допомогою бібліотеки Pydantic v2. Опанувати парадигму асинхронного програмування за допомогою ключових слів async та await, модуля стандартної бібліотеки asyncio, створення конкурентних завдань asyncio.Task, паралельної агрегації результатів через asyncio.gather, встановлення часових лімітів (timeout), реалізації стратегій повторних спроб (retry з exponential backoff) та обмеження одночасних операцій за допомогою семафорів (asyncio.Semaphore). Опанувати взаємодію з REST API через бібліотеку HTTPX (httpx.AsyncClient) та інтеграцію API із шаром збереження даних (Persistence Layer) попередньої лабораторної роботи.

2. ПОСТАНОВКА ЗАДАЧІ (ВАРІАНТ № 15)

Згідно з індивідуальним варіантом № 15 предметною областю є: «Облік результатів наукових експериментів».

Основні ресурси предметної області: дослідники (researchers) та наукові експерименти (experiments).

Вимоги до функціоналу та структури програми:

• Побудувати REST API на базі FastAPI із суворим дотриманням стандартів HTTP methods, статус-кодів та JSON;

• Реалізувати обов'язкові endpoints:

- GET /experiments — отримання списку експериментів із підтримкою пагінації, фільтрації за числовим діапазоном результатів (min_result, max_result), параметром, дослідником та сортуванням за датою (sort_by_date: asc/desc);

- GET /experiments/{id} — отримання експерименту за числовим ID (повернення 404 у разі відсутності);

- POST /experiments — створення нового запису експерименту (статус 201 Created) із повною валідацією полів та перевіркою існування дослідника;

- PATCH /experiments/{id} — часткове оновлення результату або приміток (статус 200 OK);

- DELETE /experiments/{id} — видалення експерименту (статус 204 No Content);

- GET /researchers/{id}/experiments — отримання переліку експериментів конкретного дослідника;

- GET /experiments/statistics — обчислення загальної або фільтрованої статистики (count, average, min, max);

- Додаткові endpoints керування дослідниками: GET /researchers, POST /researchers, GET /researchers/{id}, DELETE /researchers/{id}, GET /researchers/{id}/statistics;

• Забезпечити валідацію за допомогою схем Pydantic v2 (заборона порожніх рядків, перевірка фізичних меж: заборона температури нижче абсолютного нуля -273.15 °C);

• Індивідуальне асинхронне завдання варіанта № 15: реалізувати паралельне отримання статистики для декількох дослідників/наборів експериментів за допомогою Task, gather, Semaphore, HTTPX, timeout та retry;

• Розробити вичерпний набір інтеграційних та асинхронних API-тестів за допомогою pytest, pytest-asyncio та httpx.ASGITransport.

3. ТЕОРЕТИЧНІ ВІДОМОСТІ

Протокол HTTP (Hypertext Transfer Protocol) є основою взаємодії в сучасному вебі. Клієнт-серверна модель передбачає відправлення клієнтом HTTP-запиту, що містить стартовий рядок (метод, шлях, версія), заголовки (Headers) та опціональне тіло (Body), і повернення сервером HTTP-відповіді зі статус-кодом, заголовками та тілом даних (найчастіше у форматі JSON).

Архітектурний стиль REST (Representational State Transfer) ґрунтується на концепції ресурсів, що ідентифікуються унікальними URI. Для маніпуляцій ресурсами застосовуються стандартні семантичні методи HTTP: GET (читання), POST (створення), PUT (повне оновлення), PATCH (часткове оновлення) та DELETE (видалення).

Фреймворк FastAPI поєднує високу продуктивність завдяки ASGI-серверу Uvicorn, декларативну валідацію через Pydantic та автоматичну генерацію інтерактивної документації OpenAPI (Swagger UI та ReDoc).

Асинхронне програмування в Python засноване на циклі подій (Event Loop) та механізмі спільних підпрограм (Coroutines), які керуються ключовими словами async def та await. На відміну від блокуючого вводу-виводу, асинхронні операції повертають керування в Event Loop на час очікування мережевої відповіді, що дозволяє одному потоку процесора обслуговувати тисячі одночасних з'єднань. Для конкурентного виконання кількох асинхронних дій використовуються об'єкти asyncio.Task та функція asyncio.gather(). Для

запобігання перевантаженню серверів застосовуються семафори asyncio.Semaphore, що лімітують кількість активних запитів, а також стратегії Timeout та Retry з експоненційною затримкою (exponential backoff).

4. СТРУКТУРА ПРОЄКТУ

Проєкт організовано відповідно до стандарту src-layout:

```
Lab_08/
├── pyproject.toml              # Метадані пакета, залежності, налаштування
pytest та coverage
├── README.md                   # Детальна інструкція зі встановлення,
конфігурації та тестування
├── data/                       # Директорія локальної бази даних SQLite
│   └── experiments.db          # База даних для продакшн/локального режиму
├── report/                     # Офіційний звіт з лабораторної роботи
│   └── ПП_ЛАБ8_ФРАНКОВ.docx    # Офіційний документ звіту
├── tests/                      # Автоматизований тестовий набір (13 тестів)
│   ├── __init__.py
│   ├── conftest.py             # Фікстури in-memory SQLite, ізольовані сесії
та AsyncClient
│   ├── test_api.py             # Інтеграційні тести REST API (CRUD, валідація,
сортування, фільтри)
│   ├── test_async_client.py    # Тести HTTPX клієнта, Semaphore, gather,
timeout, retry
│   └── test_main.py            # Тести CLI точки входу та ініціалізації бази
даних
└── src/
    └── experiment_manager/
        ├── __init__.py         # Ініціалізація пакета
        ├── database.py         # SQLAlchemy engine, sessionmaker, get_db
FastAPI dependency
        ├── models.py           # Декларативні моделі Researcher та Experiment
        ├── schemas.py          # Схеми Pydantic v2 (валидація, серіалізація,
агрегації)
        ├── repositories.py     # Репозиторії ResearcherRepository та
ExperimentRepository
        ├── api.py              # Головний FastAPI сервіс із повним переліком
REST endpoints
        ├── async_client.py     # Асинхронний клієнт HTTPX (Task, gather,
Semaphore, timeout, retry)
        └── main.py             # Точка входу для запуску Uvicorn сервера
```

5. ОПИС ТА СПЕЦИФІКАЦІЯ REST API

У системі реалізовано наступні endpoints:

Метод HTTP	Шлях (Endpoint)	Призначення	HTTP Статус
GET	/researchers	Отримання списку дослідників (пагінація skip, limit)	200 OK
POST	/researchers	Створення нового запису дослідника	201 Created
GET	/researchers/{id}	Отримання даних дослідника за ідентифікатором	200 OK / 404
DELETE	/researchers/{id}	Видалення дослідника та каскадно його експериментів	204 No Content / 404
GET	/researchers/{id}/experiments	Отримання переліку експериментів конкретного дослідника	200 OK / 404
GET	/researchers/{id}/statistics	Статистика вимірювань для конкретного дослідника	200 OK / 404
POST	/researchers/batch-statistics	Конкурентна агрегація статистики для списку дослідників	200 OK / 504
GET	/experiments	Список експериментів із фільтрами результату, параметру та дати	200 OK

POST	/experiments	Створення експерименту з повною валідацією	201 Created / 422 / 404
GET	/experiments/{id}	Отримання конкретного експерименту за ID	200 OK / 404
PATCH	/experiments/{id}	Часткове оновлення результату або приміток	200 OK / 404
DELETE	/experiments/{id}	Видалення запису експерименту	204 No Content / 404
GET	/experiments/statistics	Глобальна або фільтрована за параметром статистика	200 OK

6. PYDANTIC СХЕМИ ТА ВАЛІДАЦІЯ ДАНИХ

Для забезпечення безпеки та цілісності даних використано Pydantic v2. Описано суворі валідатори на рівні полів (@field_validator):

```
class ExperimentBase(BaseModel):
    title: str = Field(..., min_length=3, max_length=200)
    parameter: str = Field(..., min_length=1, max_length=100)
    result: float
    date: dt.date
    notes: Optional[str] = Field(None, max_length=1000)

    @field_validator("title", "parameter")
    @classmethod
    def text_not_blank(cls, v: str) -> str:
        stripped = v.strip()
        if not stripped:
            raise ValueError("Field cannot be empty or contain only
whitespace")
        return stripped

    @field_validator("result")
    @classmethod
    def validate_result_physical_bounds(cls, v: float, info: ValidationInfo) ->
float:
```

```
        param = info.data.get("parameter", "") if info.data else ""
        if isinstance(param, str) and any(kw in param.lower() for kw in
("temp", "темп")):
            if v < -273.15:
                raise ValueError(f"Temperature {v} °C cannot be below absolute
zero (-273.15 °C)")
        return v
```

7. АСИНХРОННИЙ КЛІЄНТ, CONCURRENCY, TIMEOUT, RETRY ТА SEMAPHORE

Асинхронний клієнт (модуль async_client.py) реалізує всі ключові механізми надійного мережевого програмування:

1. Concurrency (asyncio.create_task та asyncio.gather): паралельний запуск HTTP-запитів для списку дослідників, що суттєво скорочує час агрегації статистики;

2. Rate Limiting (asyncio.Semaphore): лімітування кількості одночасно виконуваних з'єднань для захисту сервера від вичерпання пулу сокетів;

3. Retry з Exponential Backoff: автоматичне повторення запитів у разі тимчасових мережевих збоїв чи серверних помилок 5xx із формулою затримки backoff_factor * (2 ** (attempt - 1));

4. Timeout: переривання очікування за допомогою asyncio.wait_for() та таймаутів HTTPX для запобігання зависанню клієнта.

```
async def fetch_multiple_researchers_concurrently(
    researcher_ids: List[int],
    base_url: str = "http://127.0.0.1:8000",
    max_concurrency: int = 3,
    timeout: float = 5.0,
    retries: int = 2,
    client: Optional[httpx.AsyncClient] = None,
) -> BatchStatisticsResponse:
    start_time = time.perf_counter()
    semaphore = asyncio.Semaphore(max_concurrency)
```

```
    async def _run_fetch(active_client: httpx.AsyncClient):
        tasks = [
            asyncio.create_task(
                fetch_researcher_statistics_limited(
                    semaphore=semaphore, client=active_client,
                    researcher_id=r_id, base_url=base_url,
                    retries=retries, timeout=timeout
                )
            ) for r_id in researcher_ids
        ]
        raw_results = await asyncio.gather(*tasks, return_exceptions=True)
        # Обробка успішних та помилкових результатів...
```

8. РЕЗУЛЬТАТИ ВИКОНАННЯ ТА ТЕСТУВАННЯ

Автоматизоване тестування здійснено за допомогою pytest, pytest-asyncio та pytest-cov. Усі 13 розроблених тестів виконано успішно.

```
============================= test session starts ==============================
platform win32 -- Python 3.13.12, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\frankov\Documents\projects\labs\PPython\Lab_08
configfile: pyproject.toml
plugins: anyio-4.15.1, asyncio-1.4.1, cov-7.1.0
asyncio: mode=Mode.AUTO, asyncio_default_fixture_loop_scope=function
collected 13 items

Lab_08/tests/test_api.py::test_researcher_lifecycle_and_crud PASSED           [
7%]
Lab_08/tests/test_api.py::test_experiment_crud_validation_and_cascade PASSED [
15%]
Lab_08/tests/test_api.py::test_experiment_filtering_sorting_and_statistics
PASSED [ 23%]
Lab_08/tests/test_api.py::test_batch_statistics_endpoint PASSED              [
30%]
Lab_08/tests/test_async_client.py::test_fetch_with_retry_success PASSED       [
38%]
Lab_08/tests/test_async_client.py::test_fetch_with_retry_transient_failures_the
n_success PASSED [ 46%]
Lab_08/tests/test_async_client.py::test_fetch_with_retry_exhausted_raises
PASSED [ 53%]
Lab_08/tests/test_async_client.py::test_async_client_concurrency_and_semaphore
PASSED [ 61%]
Lab_08/tests/test_async_client.py::test_async_client_get_single_researcher_and_
internal_client PASSED [ 69%]
Lab_08/tests/test_async_client.py::test_async_client_timeout PASSED           [
76%]
Lab_08/tests/test_main.py::test_init_db PASSED                                [
84%]
```

```
Lab_08/tests/test_main.py::test_main_run_server PASSED                   [
92%]
Lab_08/tests/test_main.py::test_main_cli_execution PASSED
[100%]

================================ tests coverage ================================
Name                                                Stmts   Miss Branch BrPart
Cover
-------------------------------------------------------------------------------
--
Lab_08\src\experiment_manager\__init__.py               1      0      0      0
100%
Lab_08\src\experiment_manager\api.py                  108      8     22      0
92%
Lab_08\src\experiment_manager\async_client.py          79     11     20      7
82%
Lab_08\src\experiment_manager\database.py              26      4      0      0
85%
Lab_08\src\experiment_manager\main.py                  12      3      2      1
71%
Lab_08\src\experiment_manager\models.py                28      2      0      0
93%
Lab_08\src\experiment_manager\repositories.py          86      2     24      2
96%
Lab_08\src\experiment_manager\schemas.py               93     10     18      4
82%
-------------------------------------------------------------------------------
--
TOTAL                                                 433     40     86     14
88%
============================== 13 passed in 0.55s ==============================
```

9. ІСТОРІЯ КОМІТІВ GIT

Розробку виконано поетапно з фіксацією змістовних логічних комітів англійською мовою:

```
4a80901 Add comprehensive API integration and async test suite
01b2483 Add async HTTP client with concurrency, timeout, retry, and semaphore
0f64b76 Implement FastAPI application with REST endpoints
2308f9a Add Pydantic API schemas and domain models
c458f18 Setup project structure and configuration for Lab 08
```

10. ВІДПОВІДІ НА КОНТРОЛЬНІ ЗАПИТАННЯ

1. Що таке комп'ютерна мережа?

Комп'ютерна мережа — це сукупність взаємоз'єднаних обчислювальних пристроїв (комп'ютерів, серверів, мережевого обладнання), які можуть обмінюватися даними та спільно використовувати ресурси за допомогою стандартних мережевих протоколів (таких як TCP/IP).

2. Що таке клієнт-серверна архітектура?

Клієнт-серверна архітектура — це модель розподіленої системи, в якій функціональні обов'язки розділені між постачальниками послуг (серверами) та споживачами послуг (клієнтами). Клієнт формує та надсилає запит, а сервер обробляє його, звертається до сховища даних та надсилає відповідь.

3. Що таке HTTP?

HTTP (Hypertext Transfer Protocol) — прикладний мережевий протокол передачі даних без збереження стану (stateless), який функціонує за моделлю «запит-відповідь» і є стандартом для передачі веб-сторінок, файлів та даних веб-сервісів (REST API).

4. Що містить HTTP request?

HTTP-запит складається зі стартового рядка (HTTP-метод, URI ресурсу, версія протоколу), набору заголовків (Headers: Host, Content-Type, Authorization тощо), порожнього рядка-розділювача та опціонального тіла запиту (Request Body), наприклад у форматі JSON.

5. Що містить HTTP response?

HTTP-відповідь складається зі статусного рядка (версія протоколу, цифровий статус-код, текстовий опис), заголовків відповіді (Content-Type,

Content-Length тощо), порожнього рядка та тіла відповіді (Response Body) з переданими даними або повідомленням про помилку.

6. Для чого використовується GET?

HTTP-метод GET призначений виключно для безпечного та ідемпотентного отримання (читання) даних про ресурси без внесення змін до стану системи.

7. Для чого використовується POST?

HTTP-метод POST використовується для створення нового підпорядкованого ресурсу на сервері або передачі даних на обробку. Метод не є ідемпотентним: повторні однакові виклики створюють дублікати ресурсів.

8. Для чого використовується PUT?

HTTP-метод PUT використовується для повного оновлення існуючого ресурсу або його створення за точно вказаним URI, якщо він ще не існує. Метод є ідемпотентним: багаторазове виконання з однаковим тілом призводить до однакового кінцевого стану ресурсу.

9. Для чого використовується PATCH?

HTTP-метод PATCH застосовується для часткової модифікації існуючого ресурсу, коли клієнт передає лише ті поля, які необхідно змінити (наприклад, оновлення тільки результату або коментаря).

10. Для чого використовується DELETE?

HTTP-метод DELETE використовується для видалення ресурсу за вказаним ідентифікатором (URI). Метод є ідемпотентним.

11. Що означає status code 200?

Статус-код 200 OK вказує на успішне виконання запиту. Найчастіше повертається при успішних операціях GET, PUT або PATCH із передачею запитаних даних у тілі відповіді.

12. Що означає 201?

Статус-код 201 Created сигналізує про успішне створення нового ресурсу в результаті виконання запиту (типово POST-запиту). Часто супроводжується заголовком Location або тілом зі створеним об'єктом.

13. Що означає 204?

Статус-код 204 No Content означає, що запит оброблено успішно, проте тіло відповіді порожнє. Зазвичай повертається після успішної операції DELETE.

14. Що означає 404?

Статус-код 404 Not Found вказує на те, що запитаний клієнтом ресурс за вказаним URI не знайдено на сервері.

15. Що означає 422?

Статус-код 422 Unprocessable Entity сигналізує про синтаксично коректний запит (наприклад, валідний JSON), який проте містить семантичні помилки валідації полів (типово генерується FastAPI/Pydantic при порушенні обмежень типів чи валідаторів).

16. Що таке JSON?

JSON (JavaScript Object Notation) — текстовий формат обміну структурованими даними, що складається з пар «ключ-значення» та впорядкованих списків. Він є загальноприйнятим стандартом для веб-сервісів REST завдяки компактності та легкості парсингу.

17. Що таке REST?

REST (Representational State Transfer) — архітектурний стиль побудови розподілених застосунків, що базується на клієнт-серверній моделі, відсутності збереження стану між запитами (Stateless), кешованості, єдиному інтерфейсі взаємодії та маніпулюванні ресурсами через стандартні HTTP-методи.

18. Що таке resource у REST?

Ресурс у REST — це будь-яка іменована сутність або концепт предметної області (наприклад, дослідник, експеримент, замовлення), до якого можна звернутися за унікальним URI.

19. Що таке endpoint?

Endpoint (кінцева точка) — це конкретний URI у поєднанні з HTTP-методом, за яким доступний сервіс або функціонал веб-застосунку (наприклад, POST /experiments або GET /researchers/{id}).

20. Що таке path parameter?

Path parameter (параметр шляху) — змінна частина URL-адреси, яка вбудована безпосередньо в маршрут і зазвичай ідентифікує конкретний ресурс, наприклад {id} у маршруті /experiments/{id}.

21. Що таке query parameter?

Query parameter (параметр рядка запиту) — необов'язкові або фільтруючі параметри, що додаються після знака питання в URL (?parameter=Temperature&min_result=10) і слугують для фільтрації, сортування та пагінації списків.

22. Що таке request body?

Request body (тіло запиту) — блок корисного навантаження (payload), що передається разом із HTTP-запитом методів POST, PUT чи PATCH і містить структуровані дані для створення або модифікації ресурсів.

23. Що таке FastAPI?

FastAPI — це сучасний, швидкий (високопродуктивний) веб-фреймворк для мови Python, розроблений для побудови API на основі стандартних підказок типів (Python type hints), OpenAPI та Pydantic v2.

24. Для чого використовується Uvicorn?

Uvicorn — це блискавичний ASGI-сервер (Asynchronous Server Gateway Interface), побудований на uvloop та httptools, який виконує запуск та асинхронну диспетчеризацію запитів для фреймворків FastAPI та Starlette.

25. Для чого потрібний Swagger UI?

Swagger UI — це інтерактивний браузерний інтерфейс, який автоматично генерується FastAPI на основі схеми OpenAPI (/docs) і дозволяє розробникам переглядати документацію всіх ендпоінтів та надсилати тестові HTTP-запити безпосередньо з веб-сторінки.

26. Що таке Pydantic?

Pydantic — бібліотека для синтаксичного аналізу, валідації та серіалізації даних у Python, яка спирається на анотації типів мови та гарантує відповідність вхідних даних заданій схемі.

27. Що таке validation?

Validation (валідація) — процес автоматичної перевірки коректності, повноти, відповідності типам та допустимим діапазонам значень вхідних даних перед передачею їх до внутрішньої бізнес-логіки застосунку.

28. Що таке response model?

Response model у FastAPI — це Pydantic-схема, яка визначає очікувану структуру вихідних даних ендпоінта, автоматично фільтрує внутрішні/секретні атрибути об'єктів та генерує відповідну OpenAPI-документацію.

29. Що таке synchronous programming?

Синхронне програмування — модель виконання коду, за якої інструкції виконуються послідовно одна за одною. Кожна операція вводу-виводу (I/O) повністю блокує потік виконання процесора до свого завершення.

30. Що таке asynchronous programming?

Асинхронне програмування — модель однопотокового конкурентного виконання, де тривалі операції вводу-виводу (I/O) не блокують головний потік: під час очікування відповіді керування повертається в цикл подій для обслуговування інших задач.

31. Для чого використовується async?

Ключове слово async застосовується у виразі async def для оголошення корутини (coroutine) — функції, виконання якої можна призупиняти та відновлювати.

32. Для чого використовується await?

Ключове слово await використовується виключно всередині корутин для призупинення їх виконання доти, доки очікуваний об'єкт (інша корутина або Task) не завершить свою роботу й не поверне результат.

33. Що таке coroutine?

Coroutine (корутина) — це спеціальний об'єкт у Python, створений викликом async def функції, який інкапсулює стан виконання та може призупиняти свою роботу за допомогою await без блокування операційної системи.

34. Що таке event loop?

Event loop (цикл подій) — центральний диспетчер в asyncio, який керує чергою виконання корутин, реєструє події сокетів/таймерів та відновлює виконання призупинених задач, коли для них надходять готові дані.

35. Що таке Task?

Task — це об'єкт asyncio, який обгортає корутину та реєструє її для негайного виконання в циклі подій як незалежну конкурентну одиницю роботи.

36. Для чого використовується asyncio.create_task()?

Функція asyncio.create_task() використовується для планування одночасного (конкурентного) запуску корутини у фоновому режимі всередині поточного циклу подій, повертаючи екземпляр asyncio.Task.

37. Для чого використовується asyncio.gather()?

Функція asyncio.gather() використовується для одночасного паралельного очікування завершення групи корутин чи задач, збираючи їх результати у вихідний впорядкований список.

38. Що таке timeout?

Timeout (таймаут) — максимальний ліміт часу, що виділяється на виконання операції (наприклад, мережевого HTTP-запиту). Якщо операція не завершується у встановлений термін, генерується виняток (TimeoutError/TimeoutException).

39. Для чого потрібний retry?

Retry (повторна спроба) — шаблон стійкості, який автоматично повторює виконання операції у разі виникнення тимчасових (transient) помилок зв'язку або перевантаження сервера.

40. Що таке backoff?

Backoff — це алгоритм розрахунку затримки між послідовними повторними спробами запиту. Найчастіше використовують експоненційний backoff (exponential backoff), коли інтервал подвоюється після кожної невдалої спроби, запобігаючи лавинному перевантаженню цільового сервісу.

41. Для чого використовується Semaphore?

asyncio.Semaphore — примітив синхронізації, що обмежує максимальну кількість корутин, які можуть одночасно виконувати певну ресурсомістку операцію (наприклад, надсилати HTTP-запити до зовнішнього сервісу).

42. Що таке HTTPX?

HTTPX — це повнофункціональний сучасний HTTP-клієнт для Python 3, який надає синхронний та асинхронний інтерфейси (httpx.AsyncClient), підтримує HTTP/1.1 та HTTP/2, пулінг з'єднань і роботу з ASGI-застосунками.

43. Чим httpx.AsyncClient відрізняється від синхронного клієнта?

httpx.AsyncClient використовує неблокуючий асинхронний ввід-вивід на базі anyio/asyncio, інтегруючись у загальний event loop без блокування потоку на час мережевих затримок.

44. Чому AsyncClient бажано повторно використовувати?

Повторне використання екземпляра AsyncClient дозволяє утилізувати механізм Connection Pooling (пул відкритих TCP-з'єднань), усуваючи накладні витрати на повторне встановлення TCP-handshake та узгодження TLS-сертифікатів для кожного окремого запиту.

45. Навіщо обмежувати паралелізм?

Обмеження паралелізму необхідне для запобігання вичерпанню сокетів операційної системи (file descriptors), перевантаженню процесора/пам'яті клієнта та спрацьовуванню систем Rate Limiting (помилки 429 Too Many Requests) на стороні сервера.

46. Чому не слід запускати тисячі tasks одночасно?

Одночасний запуск тисяч задач може призвести до перевищення системного ліміту відкритих дескрипторів файлів/сокетів, падіння продуктивності event loop через надмірне перемикання контексту та блокування IP-адреси мережевими фаєрволами.

47. Як REST API взаємодіє з Repository Layer?

Маршрутизатор FastAPI отримує HTTP-запит, валідує параметри через Pydantic, за допомогою ін'єкції залежностей (Depends(get_db)) отримує активну сесію бази даних та делегує всі операції вибірки, фільтрації й збереження методам репозиторію (ResearcherRepository, ExperimentRepository).

48. Як FastAPI працює з Pydantic?

FastAPI використовує Pydantic як фундаментальний рушій: типи аргументів функцій-обробників автоматично співставляються зі схемами Pydantic для десеріалізації та валідації вхідних даних (JSON body, query, path), а response_model відповідає за фільтрацію та серіалізацію повернених даних у вихідний JSON.

49. Як протестувати REST API?

Тестування REST API здійснюється за допомогою бібліотеки pytest та тестового клієнта httpx.AsyncClient із використанням транспорту httpx.ASGITransport(app=app). Це дозволяє тестувати всі маршрути та статус-коди в пам'яті без реального запуску мережевого сервера.

50. Чим API integration test відрізняється від unit test?

Unit-тест ізольовано перевіряє поведінку однієї окремої функції чи класу, ізолюючи всі зовнішні залежності через Mock-об'єкти. API integration test перевіряє наскрізну взаємодію всього ланцюжка компонентів системи: від парсингу HTTP-запиту, валідації Pydantic та маршрутизації до взаємодії з базою даних і серіалізації відповіді.

11. ВИСНОВОК

У процесі виконання лабораторної роботи № 8 було успішно спроєктовано та реалізовано повноцінний асинхронний REST API сервіс для обліку результатів наукових експериментів (Варіант № 15) за допомогою сучасного стеку технологій: FastAPI, Pydantic v2 та SQLAlchemy 2.0 ORM.

У результаті роботи:

1. Реалізовано повний набір REST endpoints для керування сутностями дослідників та експериментів із підтримкою CRUD-операцій, фільтрації за числовим діапазоном, сортування за датою та обчислення статистичних показників;

2. Забезпечено строгу декларативну валідацію вхідних даних засобами Pydantic v2, включаючи перевірку фізичних меж вимірювань (заборона температури нижче абсолютного нуля);

3. Створено та досліджено клієнт асинхронної взаємодії на базі HTTPX з паралельним виконанням запитів через asyncio.Task та asyncio.gather, контролем конкурентності через asyncio.Semaphore, таймаутами та механізмом повторних спроб (retry з exponential backoff);

4. Розроблено надійний набір інтеграційних та асинхронних автоматизованих тестів на базі pytest та httpx.ASGITransport, що забезпечив 100% успішність проходження тестів (13/13) та 88% branch coverage;

5. Вся розробка зафіксована у системі контролю версій Git відповідно до вимог поетапної розробки.